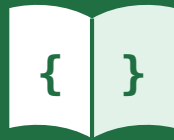

Modül 19 — Native Image ve Performans

Ödevler

Spring Boot Edu



2026-09-25

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Reflection Kullanan Kodu Native'e Hazırlamak (Orta)	2
Ödev 2 — Başlangıcı Hangi Bean'ler Yavaşlatıyor? (Orta)	3
Ödev 3 — Ölçün ve Tabloyu Doldurun (Kolay)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/19-native-performance/exercise/` altındadır. TODO yorumlarını bulun.
2. Ödev 1 ve 2 ne Docker ne de GraalVM ister: Testler ipuçlarını ve başlangıç adımlarını JVM’de kontrol eder. Ödev 3 ders uygulamasıyla bir ölçümdür.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/19-native-performance/exercise -am test
```

4. Tüm testler yeşil olduğunda (ve ödev 3 için tablo doldurulduğunda) ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/19-native-performance/solution/` altındaki çözümü okuyun.

Ödev 1 — Reflection Kullanan Kodu Native’e Hazırlamak (Orta)

Hedef: `ExportService` JVM’de çalışıyor. Bir native image’da üç şey eksik olurdu: Sınıf adıyla oluşturulan export formatı, `export/header.txt` dosyası ve `ImportedBook`’un JSON bağlaması. İpuçlarını ekleyin.

Yapılacaklar (exercisel paketi):

- TODO 1a (`ExportHints`) — `CsvExport` ve `MarkdownExport`’un reflection ile oluşturulmasına izin verin ve `export/header.txt`’yi dahil edin.
- TODO 1b (`ExportService`) — `ExportHints`’i servis için kaydedin.
- TODO 1c (`ExportService`) — Spring’e `ImportedBook`’un JSON’dan bağlandığını söyleyin.

İpuçları:

- Ders bölüm 3.4: `hints.reflection().registerType(..., MemberCategory.INVOKE_DECLARED_CONSTRUCTORS)` ve `hints.resources().registerPattern(...)`.
- `@ImportRuntimeHints(ExportHints.class)` herhangi bir bean sınıfında çalışır.
- `@RegisterReflectionForBinding(ImportedBook.class)`, Jackson’ın ihtiyaç duyduğunu (constructor, accessor’lar) kaydeder.
- `Exercise1Test`, `process-aot`’un build sırasında yaptığı gibi servis için Spring’in gerçek AOT işlemlerini (`ApplicationContextAotGenerator`) çalıştırır.

Kabul kriterleri: `Exercise1Test` içindeki 3 testin hepsi geçer.

Tahmini süre: 25 dakika

Ödev 2 — Başlangıcı Hangi Bean'ler Yavaşlatıyor? (Orta)

Hedef: Uygulama başlangıç adımlarını `BufferingApplicationStartup` ile kaydeder (bkz. `NativePerformanceApplication`). En yavaş bean oluşturmalarının bir raporunu yazın. `SlowCatalogWarmup` bilerek 300 ms sürer.

Yapılacaklar (`exercise2.StartupReport.slowestBeans`):

- TODO 2a — `spring.beans.instantiate` adlı kaydedilmiş adımları alın.
- TODO 2b — Her birini bir `BeanTiming`'e çevirin. Bean adı, adımın `beanName` etiketidir.
- TODO 2c — En yavaşlar önce olacak şekilde sıralayın ve en fazla limit kadarını döndürün.

İpuçları:

- `startup.getBufferedTimeline().getEvents()`, `getStartupStep()` ve `getDuration()` içeren `TimelineEvent`'ler döndürür.
- `event.getStartupStep().getTags()`, `getKey()` ve `getValue()` içeren bir `Iterable<StartupStep.Tag>`'dir.
- `Comparator.comparing(BeanTiming::duration).reversed()`.
- `Actuator` ile aynı veri `/actuator/startup`'ta bulunur.

Kabul kriterleri: `Exercise2Test` geçer.

Tahmini süre: 25 dakika

Ödev 3 — Ölçün ve Tabloyu Doldurun (Kolay)

Hedef: AOT işlemenin, AOT cache'in ve bir native image'ın neyi değiştirdiğini kendi sayılarınızla görün.

Yapılacaklar:

1. PostgreSQL'i başlatın (`docker compose --profile postgres up -d`) ve native image'ı bir kez build edin (ders bölüm 3.5).
2. `modules/19-native-performance/compare-startup.sh` 5'i çalıştırın.
3. Sayılarınızı tabloya kopyalayın ve soruları cevaplayın.

Varyant	Başlangıç ms	RSS MB
JVM		
JVM + Spring AOT		
JVM + Spring AOT + AOT cache		
Native image		

- Başlangıç süresinde en büyük kazancı hangi adım sağlıyor? Bellekte hangisi?
- AOT cache varyantını eğitim çalıştırması olmadan çalıştırın (target/startup/app.aot'u silin). Ne olur?
- Native image ile neyden vazgeçersiniz (build süresi, tepe performansı, reflection)? Ne zaman değer?

Kabul kriterleri: Tablo doldurulmuştur ve her sorunun iki cümlelik bir cevabı vardır.

Tahmini süre: 30 dakika (artı native build)

Ek Meydan Okuma (İsteğe Bağlı)

Derse spring-boot-starter-actuator ekleyin, /actuator/startup'ı açın ve çıktısını kendi StartupReport'unuzla karşılaştırın. Ardından native image'ı actuator ile build edin ve endpoint'in hâlâ çalışıp çalışmadığını kontrol edin.